Fine-tuning SR3 on endoscopy images under a power-transformed noise schedule

Phase 2 report, Foundation Models and Generative AI

Group	[GROUP NUMBER / NAME - to be filled in]
Members	Rikin Patel (M26AI1096); Anshul (M26AI1020); Sonalika Sharma (M26AI1120); Balasubramanium M (M26AI1030)
Code	https://github.com/rikin-patel/sr3_diffusion, branch `feature/phase2-report`
Experiment logs	https://wandb.ai/rikin-patel-indian-institute-of-technology-jodhpur/FMGAI-Assn (Weights & Biases project FMGAI-Assn)

Summary. We take the public SR3 checkpoint for 16×16 → 128×128 super-resolution, trained on face photographs, and apply it to Kvasir-SEG colonoscopy images. Without fine-tuning (Run 0) it reaches 22.49 dB PSNR on our 100 test images. Fine-tuning for 100 epochs with the native schedule (Run A) gives 24.15 dB, and fine-tuning with the schedule raised to the power x = 2.50 (Run B) gives 23.43 dB. Both fine-tuned runs are clearly better than Run 0. Run A is ahead of Run B by 0.72 dB (paired standard error 0.15), while the perceptual score LPIPS is the same for both (0.170 and 0.171). Each run was trained once, with one seed.

1. Task and Model

Task. 8× single-image super-resolution of gastrointestinal endoscopy images, from 16×16 to 128×128 pixels. The input is a 16×16 RGB image made by bicubic downsampling of a colonoscopy frame. The output is a 128×128 RGB image, and the target is the original frame resized and centre-cropped to 128×128.

Model. SR3 (Saharia et al., "Image Super-Resolution via Iterative Refinement", arXiv:2104.07636) is a conditional denoising diffusion model. The authors did not release weights, so we use the open PyTorch implementation by Janspiry (https://github.com/Janspiry/Image-Super-Resolution-via-Iterative-Refinement, pinned at commit `01d27a7`) and its released 16×16 → 128×128 checkpoint `I640000_E37` (README.md, line 84). The network is a U-Net with 97.8 million parameters. It receives the low-resolution image, upsampled to 128×128 with bicubic interpolation, concatenated channel-wise with the current noisy image.

Forward process. Let x_0 be the clean 128×128 image, c the bicubic-upsampled conditioning image, $\varepsilon \sim N(0, I)$ Gaussian noise, and $t = 1, \ldots, T$ the timestep with $T = 2{,}000$. The per-step noise variances are β_t, increasing linearly from 10^{-6} to 10^{-2}. With $\alpha_t = 1 - \beta_t$, the forward process adds noise step by step, $q(x_t \mid x_{t-1}) = N(\sqrt{\alpha_t} \cdot x_{t-1}, \beta_t I)$, which gives the closed form

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon, \quad \text{with} \quad \bar{\alpha}_t = \prod_{s=1..t} \alpha_s = \prod_{s=1..t} (1 - \beta_s).$$

The network $\varepsilon_\varphi(c, x_t, \sqrt{\bar{\alpha}_t})$ predicts the noise ε and is trained with an L1 loss, $\|\varepsilon - \varepsilon_\varphi(c, x_t, \sqrt{\bar{\alpha}_t})\|_1$. Sampling starts from $x_T \sim N(0, I)$ and applies the learned reverse step T times.

Native noise parameterization. We take $\boldsymbol{\theta_t = \bar{\alpha}_t}$, the cumulative product of the per-step signal-retention factors. The SR3 paper calls it γ_t ("$\gamma_t = \prod_{i=1..t} \alpha_i$", Section 2.1, page 3) and states "Our model conditions on γ directly (vs t as in [17])" (Section 2.4, page 4). The code does the same: the forward sample is built from $\bar{\alpha}_t$ alone and the network receives $\sqrt{\bar{\alpha}_t}$ instead of the timestep index (diffusion.py, lines 95-106, 153-157, 212-231 at the pinned commit). The config specifies the schedule through β_t, and $\bar{\alpha}_t$ follows from it deterministically; we chose $\bar{\alpha}_t$ because it is the quantity both the forward process and the network actually consume.

2. Dataset and Verification

Dataset. Kvasir-SEG (Jha et al., "Kvasir-SEG: A Segmented Polyp Dataset", MultiMedia Modeling 2020, arXiv:1911.07069), downloaded from https://datasets.simula.no/kvasir-seg/. It contains **1,000 colonoscopy images** of polyps, each with a segmentation mask. All 1,000 files are RGB JPEGs. They come in 333 different resolutions, with widths from 332 to 1920 pixels and heights from 352 to 1072 pixels; the smallest image is 332×487 and the largest 1920×1072. We use the

images only. The masks are not needed for super-resolution.

Why it suits the task. Every image is far larger than 128×128, so a true high-resolution target exists for each low-resolution input and nothing has to be upsampled to create the ground truth. The content (mucosa, folds, polyps, specular highlights) is rich in fine texture, which is exactly what 8× super-resolution has to reconstruct. And the domain is far from the faces the model was pretrained on, so the effect of fine-tuning is easy to isolate. With 1,000 images the dataset is also small enough to fine-tune on one laptop GPU.

Evidence that it was not used in pretraining. As in the declaration, we follow option (b). Because the weights come from the open implementation, its repository is the authoritative record of what this checkpoint was trained on. Line numbers refer to commit `01d27a7`.

Source	Location	What it says
Codebase, README.md	line 24	Conditional model listed as "16×16 -> 128×128 on FFHQ-CelebaHQ".
Codebase, README.md	line 84	The released checkpoint we use is labelled "16×16 -> 128×128 on FFHQ-CelebaHQ".
Codebase, config/sr_sr3_16_128.json	lines 16-19	Training set: `"name": "FFHQ"`, `"dataroot": "dataset/ffhq_16_128"`.
Codebase, config/sr_sr3_16_128.json	lines 28-31	Validation set: `"name": "CelebaHQ"`, `"dataroot": "dataset/celebahq_16_128"`.
Paper, arXiv:2104.07636	page 5, Section 4, right column, lines 11-13 (paragraph beginning "Datasets:")	"training face super-resolution models on Flickr-Faces-HQ (FFHQ) and evaluating on CelebA-HQ".

FFHQ and CelebA-HQ are collections of aligned, centred photographs of human faces taken with ordinary cameras in visible light. Kvasir-SEG images are frames recorded inside the large bowel with a medical endoscope. They show mucosa, folds, polyps, fluid and strong specular highlights from the endoscope light, usually inside a dark border and sometimes with an overlaid position marker. There are no faces, people or everyday objects in them, and their colours (mostly reds and pinks), textures and scene geometry differ from portrait photography. The pretraining pipeline reads only the two face folders named in the config, so Kvasir-SEG could not have entered pretraining.

3. Schedule Analysis

$\bar{\alpha}_t \in (0, 1)$ for all t. Every β_s lies in $[10^{-6}, 10^{-2}]$, so every factor $1 - \beta_s$ lies strictly between 0 and 1, and a finite product of such factors is again strictly between 0 and 1. Numerically, in double precision, $\bar{\alpha}_t$ decreases monotonically from **0.999999** at t = 1 to **4.386×10^{-5}** at t = 2,000. For x = 2.50, $\bar{\alpha}_t^{x}$ decreases from **0.9999975** to **1.274×10^{-11}**, which is still strictly positive and representable in single precision. The training code asserts both bounds every time a schedule is built, and the `alphas_cumprod` buffer stored in the pretrained checkpoint equals this native schedule.

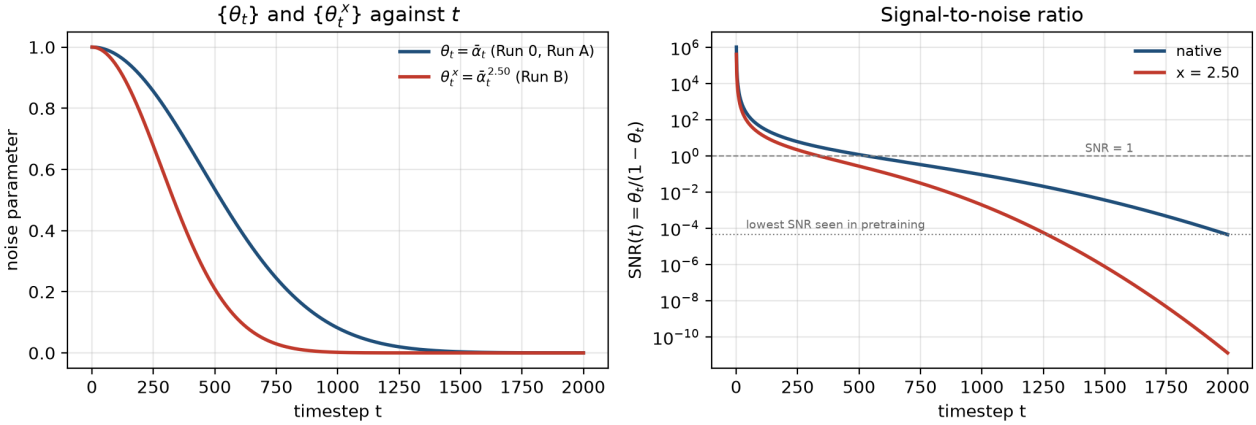


Figure 1. Left: the native schedule $\bar{\alpha}_t$ and the transformed schedule $\bar{\alpha}_t^{x}$ against t on the same axes. Right: the signal-to-noise ratio of both schedules on a log scale.

What changes in the forward process when x > 1. Replacing $\bar{\alpha}_t$ by $\bar{\alpha}_t^{x}$ gives

$$x_t = \sqrt{(\bar{\alpha}_t^{x})} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t^{x})} \cdot \varepsilon.$$

- Since $0 < \bar{\alpha}_t < 1$ and $x > 1$, $\bar{\alpha}_t^{x} < \bar{\alpha}_t$ at every t. The signal is scaled down by a further factor $\bar{\alpha}_t^{(x-1)/2}$ and the noise variance rises from $1 - \bar{\alpha}_t$ to $1 - \bar{\alpha}_t^{x}$. The image is destroyed faster.
- The transformed process is still a valid Markov diffusion. Because $(\prod \alpha_s)^{x} = \prod \alpha_s^{x}$, it has per-step factors $\alpha'_t = \alpha_t^{x}$ and variances $\beta'_t = 1 - (1 - \beta_t)^{x} \approx x \cdot \beta_t$. Each step injects about x times as much noise; the largest step variance grows from 0.010 to 0.0248.
- For this linear β schedule, $-\log \bar{\alpha}_t$ grows almost exactly like t^2, so $\bar{\alpha}_t^{x} = \bar{\alpha}_\tau$ with $\tau \approx \sqrt{x} \cdot t = 1.58\ t$. The modified process at step t is at the noise level the native process only reaches at step 1.58 t: step 200 corresponds to native step 317, step 400 to 633, and step 800 to 1265.
- As a result the modified process passes the native terminal noise level ($\bar{\alpha}_t = 4.386 \times 10^{-5}$) already at t = 1266. The remaining **735 of 2,000 timesteps (36.8%)** lie at noise levels below anything the pretrained network was trained on; its noise-level input $\sqrt{\bar{\alpha}_t}$ was never smaller than 0.0066 in pretraining and goes down to 3.57×10^{-6} in Run B.

Signal-to-noise ratio. With $\mathrm{SNR}(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$, the exponent changes it to $\mathrm{SNR}_x(t) = \bar{\alpha}_t^{x} / (1 - \bar{\alpha}_t^{x})$. The ratio of the two is

$$\mathrm{SNR}_x(t) / \mathrm{SNR}(t) = \bar{\alpha}_t^{x-1} \cdot (1 - \bar{\alpha}_t) / (1 - \bar{\alpha}_t^{x}).$$

Near $t = 1$, where $\bar{\alpha}_t \to 1$, we have $1 - \bar{\alpha}_t^{x} \approx x\,(1 - \bar{\alpha}_t)$, so the ratio tends to $1/x = 0.40$: even the cleanest steps lose a factor x in SNR. As t grows the ratio falls towards $\bar{\alpha}_t^{x-1}$, which goes to zero. The SNR is therefore lowered at every timestep, by a factor that starts at 1/x and keeps shrinking. The point where signal and noise are equal (SNR = 1) moves from t = 527 to t = 334, so the number of timesteps with more signal than noise drops from 526 to 333.

t	$\bar{\alpha}_t$ (native)	$\bar{\alpha}_t^{2.50}$	SNR (native)	SNR (x = 2.50)	ratio
1	0.9999990	0.9999975	1.000×10^{6}	4.000×10^{5}	4.00×10^{-1}
250	0.8555573	0.6770531	5.923×10^{0}	2.096×10^{0}	3.54×10^{-1}
500	0.5352504	0.2096006	1.152×10^{0}	2.652×10^{-1}	2.30×10^{-1}
1,000	0.0817838	1.913×10^{-3}	8.907×10^{-2}	1.916×10^{-3}	2.15×10^{-2}
1,500	3.556×10^{-3}	7.541×10^{-7}	3.569×10^{-3}	7.541×10^{-7}	2.11×10^{-4}
2,000	4.386×10^{-5}	1.274×10^{-11}	4.386×10^{-5}	1.274×10^{-11}	2.90×10^{-7}

Table 1. Noise parameter and signal-to-noise ratio of both schedules at selected timesteps.

4. Experimental Setup

Dataset preparation. We downloaded `kvasir-seg.zip` from the Simula page and used the `images/` folder as it is. Cleaning consisted of checks rather than removals: all 1,000 files open as RGB images, none is smaller than 128 pixels on its shorter side, and a SHA-256 comparison found no

exact duplicate files, so no image was dropped. Each image is then processed the way the SR3 codebase prepares its own data: the shorter side is resized with bicubic interpolation and the image is centre-cropped to a square. This is done three times per image, giving the 128×128 target, the 16×16 input, and the 16×16 input upsampled back to 128×128 (the conditioning image). Pixel values are scaled to [−1, 1]. The only augmentation is a random horizontal flip during training, the codebase default.

Split. 800 training, 100 validation and 100 test images. The file names are sorted, shuffled once with Python's `random.Random(42)`, and cut at 800 and 900. The three file lists are stored in `kvasir_data/splits/`. The splits are disjoint and together cover all 1,000 images.

Fine-tuning hyperparameters, identical in Run A and Run B:

Setting	Value
Starting weights	Generator weights of `I640000_E37` for both runs. The Adam state saved with the checkpoint is not loaded; both runs start a new optimizer at iteration 0.
Optimizer	Adam (PyTorch defaults, $\beta = (0.9, 0.999)$), constant learning rate 1×10^{-4}, no weight decay
Batch size	4
Length	100 epochs = 20,000 iterations (200 iterations per epoch over 800 images)
Loss	L1 between true and predicted noise
Network	U-Net exactly as in the pretrained config, dropout 0.2, all weights trainable
Diffusion steps	T = 2,000 for training and for sampling
Random seed	42 for Python, NumPy and PyTorch. With the same seed both runs see the same batches, the same timestep draws and the same noise. cuDNN auto-tuning is left on, so results are not bit-reproducible.
Only difference	Run A uses $\bar{\alpha}_t$; Run B uses $\bar{\alpha}_t^{2.50}$, for training and for sampling.

Validation during fine-tuning. Every 10 epochs (2,000 iterations) the current model super-resolves the first 20 validation images with the full 2,000-step reverse process, using the schedule of its own run and a fixed sampling seed, so every validation pass and both runs start from the same noise. We record mean PSNR and SSIM against the 128×128 ground truth and save all generated images. Validation was used for monitoring only: no hyperparameter and no checkpoint was selected with it. A checkpoint is written every 25 epochs, and the final one (epoch 100) is the model that is evaluated.

Test evaluation. The 100 test images were touched only after fine-tuning had finished. Each run super-resolves all of them once with the full 2,000-step reverse process, in batches of 25, starting from the same seeded Gaussian noise in all three runs. We report PSNR, SSIM and LPIPS (AlexNet) per image against the ground truth, and compare runs with paired per-image differences. Bicubic upsampling is included as a reference.

Hardware and software. One NVIDIA GeForce RTX 5070 Ti Laptop GPU (11.9 GB), Windows 11. Python 3.12.10, PyTorch 2.8.0+cu129 (CUDA 12.9), torchvision 0.23.0+cu129, NumPy 2.5.1, OpenCV 5.0.0.93, Pillow 12.2.0, lpips 0.1.4, tensorboardX 2.6.5, wandb 0.28.1.

5. Implementation

Files and lines changed. Six files of the original codebase were modified. Every changed line carries a `# MODIFIED` comment and every added line an `# ADDED` comment. Line numbers below refer to the submitted files.

File	Changed lines (submitted file)	Purpose
`sr.py`	1-4, 15, 17-25, 42, 45-52, 123-124, 136-143, 148-153, 155-171, 173-179, 182, 186, 188-189, 192, 197-198	Named constant `SCHEDULE_EXPONENT_X = 2.50` at the top and the function that applies it; random seed; validation in batches with a fixed sampling seed; SSIM during validation.
`model/sr3_modules/diffusion.py`	104-118, 215	The schedule change itself: $\bar{\alpha}_t$ is raised to the power x and β_t is re-derived; bound assertions; batched sampling returns the whole batch.
`model/model.py`	43-47, 99-102, 170-175	Schedule is rebuilt from the config after a checkpoint is loaded; option to start from weights only; the active schedule is written to the log.
`core/logger.py`	74-75	Number of validation images during training is configurable.
`core/wandb_logger.py`	20, 23-25	Run name, group and job type are passed to Weights & Biases.
`data/__init__.py`	17, 19-20	Validation batch size is configurable.

Eight new scripts were added under `assignment/` (data preparation, schedule analysis, test evaluation, log collection, run comparison, and three builders for the deliverables and the two PDFs) together with three run configs. The two fine-tuning configs differ only in the run name and in the flag `"use_power_schedule"`, which is false for Run A and true for Run B.

How the transformation is implemented. The original code builds β_t, then α_t and $\bar{\alpha}_t$, and derives every other buffer from them. We raise $\bar{\alpha}_t$ to the power x right after it is computed and then recompute $\beta_t = 1 - \bar{\alpha}_t^{\,x} / \bar{\alpha}_{t-1}^{\,x}$ and α_t, so that the posterior mean and variance used in sampling stay consistent with the modified forward process. The training loss needs no change, because it already draws its noise level from the $\bar{\alpha}_t$ table.

A pitfall we had to fix. The checkpoint file contains the schedule buffers of the run that wrote it, and the original training path never rebuilds them after loading. Left alone, Run B would have trained with the modified noise levels while holding native buffers. `model/model.py` now rebuilds the schedule from the config immediately after loading.

Note on the named constant. When Run A and Run B were executed, the exponent was read from the config entry `"theta_power": 2.5`. To meet the submission requirement we afterwards moved it into the constant `SCHEDULE_EXPONENT_X` at the top of `sr.py` and changed the comment markers to the required spelling. We then rebuilt both schedules through the submitted code and compared them with the buffers stored in the two fine-tuned checkpoints: they are bit-for-bit identical, so the submitted code produces exactly the schedules the runs used.

Original and modified lines, side by side. Red: original lines that were replaced or removed. Green: lines in the submitted file. Unchanged lines are omitted.

ORIGINAL sr.py (commit 01d27a7)	**MODIFIED sr.py (submitted version)**

nothing here (new lines inserted after line 0)	**lines 1-4**
	`1 # Declared exponent x of the power-transformed noise schedule {theta_t ** x}, # ADDED`
	`2 # with theta_t = alpha_bar_t. Used only when a config sets " use_power_schedule": true (Run B). # ADDED`
	`3 SCHEDULE_EXPONENT_X = 2.50 # ADDED`
	`4`

nothing here (new lines inserted after line 10)	**line 15** `15 import random # ADDED`

nothing here (new lines inserted after line 11)	**lines 17-25**
	`17`
	`18`
	`19 def apply_schedule_exponent(opt): # ADDED`
	`20 """Write the exponent into every schedule block: x for Run B, 1.0 otherwise.""" # ADDED`
	`21 for schedule_opt in opt['model']['beta_schedule'].values(): # ADDED`
	`22 use_power = bool(schedule_opt.get('use_power_schedule')) # ADDED`
	`23 schedule_opt['theta_power'] = SCHEDULE_EXPONENT_X if use_power else 1.0 # ADDED`
	`24 return opt # ADDED`

ORIGINAL sr.py (commit 01d27a7)	MODIFIED sr.py (submitted version)
	25
nothing here (new lines inserted after line 27)	**line 42**
	42 opt = apply_schedule_exponent(opt) # ADDED
nothing here (new lines inserted after line 29)	**lines 45-52**
	45 46 # Same seed for Run A and Run B: identical data order and noise draws. # ADDED 47 seed = opt['train']['seed'] if opt['train'] else None # ADDED 48 if seed is not None: # ADDED 49 random.seed(seed) # ADDED 50 np.random.seed(seed) # ADDED 51 torch.manual_seed(seed) # ADDED 52 torch.cuda.manual_seed_all(seed) # ADDED
line 100	**lines 123-124**
100 wandb_logger.log_metrics(logs)	123 wandb_logger.log_metrics(# MODIFIED (re-indented) 124 dict(logs, iteration=current_step, epoch_running=current_epoch)) # MODIFIED
nothing here (new lines inserted after line 111)	**lines 136-143**
	136 avg_ssim = 0.0 # ADDED 137 # Validation sampling uses its own fixed seed so every validation # ADDED 138 # pass (and both runs) starts from the same noise; the training # ADDED 139 # RNG stream is restored afterwards. # ADDED 140 rng_state = (torch.get_rng_state(), torch.cuda.get_rng_state_all()) # ADDED 141 if seed is not None: # ADDED 142 torch.manual_seed(seed) # ADDED 143 torch.cuda.manual_seed_all(seed) # ADDED
line 113	**removed**
113 idx += 1	
lines 117-120	**lines 148-153**
117 sr_img = Metrics.tensor2img(visuals['SR']) # uint8 118 hr_img = Metrics.tensor2img(visuals['HR']) # uint8 119 lr_img = Metrics.tensor2img(visuals['LR']) # uint8 120 fake_img = Metrics.tensor2img(visuals['INF']) # uint8	148 for b in range(visuals['HR'].shape[0]): # ADDED: validation batches may hold several images 149 idx += 1 # MODIFIED (re-indented) 150 sr_img = Metrics.tensor2img(visuals['SR'][b]) # uint8 # MODIFIED 151 hr_img = Metrics.tensor2img(visuals['HR'][b]) # uint8 # MODIFIED 152 lr_img = Metrics.tensor2img(visuals['LR'][b]) # uint8 # MODIFIED 153 fake_img = Metrics.tensor2img(visuals['INF'][b]) # uint8 # MODIFIED
lines 122-137	**lines 155-171**
122 # generation 123 Metrics.save_img(124 hr_img, '{}/{}_{}_hr.png'.format(result_path, current_step, idx)) 125 Metrics.save_img(126 sr_img, '{}/{}_{}_sr.png'.format(result_path, current_step, idx)) 127 Metrics.save_img(128 lr_img, '{}/{}_{}_lr.png'.format(result_path, current_step, idx)) 129 Metrics.save_img(130 fake_img, '{}/{}_{}_inf.png'.format(result_path, current_step, idx)) 131 tb_logger.add_image(132 'Iter_{}'.format(current_step), 133 np.transpose(np.concatenate(134 (fake_img, sr_img, hr_img), axis=1), [2, 0, 1]), 135 idx) 136 avg_psnr += Metrics.calculate_psnr(137 sr_img, hr_img)	155 # generation # MODIFIED (re-indented) 156 Metrics.save_img(# MODIFIED (re-indented) 157 hr_img, '{}/{}_{}_hr.png'.format(result_path, current_step, idx)) # MODIFIED (re-indented) 158 Metrics.save_img(# MODIFIED (re-indented) 159 sr_img, '{}/{}_{}_sr.png'.format(result_path, current_step, idx)) # MODIFIED (re-indented) 160 Metrics.save_img(# MODIFIED (re-indented) 161 lr_img, '{}/{}_{}_lr.png'.format(result_path, current_step, idx)) # MODIFIED (re-indented) 162 Metrics.save_img(# MODIFIED (re-indented) 163 fake_img, '{}/{}_{}_inf.png'.format(result_path, current_step, idx)) # MODIFIED (re-indented) 164 tb_logger.add_image(# MODIFIED (re-indented) 165 'Iter_{}'.format(current_step), # MODIFIED (re-indented) 166 np.transpose(np.concatenate(# MODIFIED (re-indented) 167 (fake_img, sr_img, hr_img), axis=1), [2, 0, 1]), # MODIFIED (re-indented) 168 idx) # MODIFIED (re-indented) 169 avg_psnr += Metrics.calculate_psnr(# MODIFIED (re-indented) 170 sr_img, hr_img) # MODIFIED (re-indented) 171 avg_ssim += Metrics.calculate_ssim(sr_img, hr_img) # ADDED
lines 139-143	**lines 173-179**
139 if wandb_logger: 140 wandb_logger.log_image(141 f'validation_{idx}', 142 np.concatenate((fake_img, sr_img, hr_img), axis=1) 143)	173 if wandb_logger and idx <= 4: # MODIFIED: only the first few panels go to W&B 174 wandb_logger.log_image(# MODIFIED (re-indented) 175 f'validation_{idx}', # MODIFIED (re-indented) 176 np.concatenate((fake_img, sr_img, hr_img), axis=1) # MODIFIED (re-indented) 177) # MODIFIED (re-indented) 178 torch.set_rng_state(rng_state[0]) # ADDED 179 torch.cuda.set_rng_state_all(rng_state[1]) # ADDED
nothing here (new lines inserted after line 145)	**line 182**
	182 avg_ssim = avg_ssim / idx # ADDED

ORIGINAL sr.py (commit 01d27a7)	MODIFIED sr.py (submitted version)
line 149	**line 186**
149 logger.info('# Validation # PSNR: {:.4e}'.format(avg_psnr))	186 logger.info('# Validation # PSNR: {:.4e} SSIM: {:.4e}'.format(avg_psnr, avg_ssim)) # MODIFIED
lines 151-152	**lines 188-189**
151 logger_val.info('<epoch:{:3d}, iter:{:8,d}> psnr: {:.4e}'.format(152 current_epoch, current_step, avg_psnr))	188 logger_val.info('<epoch:{:3d}, iter:{:8,d}> psnr: {:.4e}, ssim: {:.4e}'.format(# MODIFIED (re-indented) 189 current_epoch, current_step, avg_psnr, avg_ssim)) # MODIFIED
nothing here (new lines inserted after line 154)	**line 192**
	192 tb_logger.add_scalar('ssim', avg_ssim, current_step) # ADDED
nothing here (new lines inserted after line 158)	**lines 197-198**
	197 'validation/val_ssim': avg_ssim, # ADDED 198 'validation/iteration': current_step, # ADDED

ORIGINAL model/sr3_modules/diffusion.py (commit 01d27a7)	MODIFIED model/sr3_modules/diffusion.py (submitted version)
nothing here (new lines inserted after line 103)	**lines 104-118**
	104 # Power-transformed schedule: theta_t = alphas_cumprod (gamma_t in the SR3 paper). # ADDED 105 # theta_power = 1.0 keeps the native schedule (Run 0 / Run A); Run B uses x = 2.50. # ADDED 106 theta_power = schedule_opt['theta_power'] if 'theta_power' in schedule_opt else None # ADDED 107 theta_power = 1.0 if theta_power is None else float(theta_power) # ADDED 108 in_unit_interval = np.all((alphas_cumprod > 0.) & (alphas_cumprod < 1.)) # ADDED 109 assert in_unit_interval, 'theta_t = alphas_cumprod must lie strictly in (0, 1)' # ADDED 110 if theta_power != 1.0: # ADDED 111 alphas_cumprod = np.power(alphas_cumprod, theta_power) # ADDED 112 # betas/alphas are re-derived so that every other buffer stays consistent # ADDED 113 # with the transformed cumulative product. # ADDED 114 betas = 1. - alphas_cumprod / np.append(1., alphas_cumprod[:-1]) # ADDED 115 alphas = 1. - betas # ADDED 116 in_unit_interval = np.all((alphas_cumprod > 0.) & (alphas_cumprod < 1.)) # ADDED 117 assert in_unit_interval, 'theta_t ** x left the interval (0, 1)' # ADDED 118 self.theta_power = theta_power # ADDED
line 200	**line 215**
200 return ret_img[-1]	215 return img # MODIFIED: was ret_img[-1], which kept only the last image of a batch

ORIGINAL model/model.py (commit 01d27a7)	MODIFIED model/model.py (submitted version)
nothing here (new lines inserted after line 42)	**lines 43-47**
	43 # The checkpoint stores the schedule buffers of the run that produced it, so # ADDED 44 # loading overwrites the schedule built from the config. Rebuild it here. # ADDED 45 self.schedule_phase = None # ADDED 46 self.set_new_noise_schedule(# ADDED 47 opt['model']['beta_schedule']['train'], schedule_phase='train') # ADDED
nothing here (new lines inserted after line 93)	**lines 99-102**
	99 net = self.netG.module if isinstance(self.netG, nn.DataParallel) else self.netG # ADDED 100 logger.info('Noise schedule [{}]: theta_power={:.2f}, T={}, alpha_bar_1={:.9f}, alpha_bar_T={:.6e}'.format(# ADDED 101 schedule_phase, net.theta_power, net.num_timesteps, # ADDED 102 net.alphas_cumprod[0].item(), net.alphas_cumprod[-1].item())) # ADDED
line 161	**lines 170-175**
161 if self.opt['phase'] == 'train':	170 # Fine-tuning starts from the pretrained weights only: fresh Adam, step 0. # ADDED 171 reset_optimizer = bool(self.opt['train'] and self.opt['train']['reset_optimizer']) # ADDED 172 if reset_optimizer: # ADDED 173 logger.info('reset_optimizer=True: pretrained weights loaded, ' # MODIFIED (re-indented) 174 'optimizer state and step counter are NOT restored.') # ADDED 175 if self.opt['phase'] == 'train' and not reset_optimizer: # MODIFIED

ORIGINAL core/logger.py (commit 01d27a7)	MODIFIED core/logger.py (submitted version)
line 74	**lines 74-75**
74 opt['datasets']['val']['data_len'] = 3	74 # was hardcoded to 3; a config can now set train.val_data_len # MODIFIED 75 opt['datasets']['val']['data_len'] = opt['train'].get('val_data_len', 3) # MODIFIED

ORIGINAL core/wandb_logger.py (commit 01d27a7)	MODIFIED core/wandb_logger.py (submitted version)
nothing here (new lines inserted after line 19)	**line 20** `20  os.makedirs('./experiments', exist_ok=True) # ADDED`

nothing here (new lines inserted after line 21)	**lines 23-25**
	`23  name=opt['wandb']['run_name'] or opt['name'], # ADDED`
	`24  group=opt['wandb']['group'], # ADDED`
	`25  job_type=opt['wandb']['job_type'] or opt['phase'], # ADDED`

ORIGINAL data/__init__.py (commit 01d27a7)	MODIFIED data/__init__.py (submitted version)
nothing here (new lines inserted after line 16)	**line 17** `17  # batch size was fixed to 1; configurable so validation can sample in one batch # MODIFIED`

line 18
`18  dataset, batch_size=1, shuffle=False, num_workers=1, pin_memory=True)`
	lines 19-20
	`19  dataset, batch_size=dataset_opt.get('batch_size') or 1, # MODIFIED`
	`20  shuffle=False, num_workers=1, pin_memory=True) # MODIFIED (re-indented)`

How we verified that Run B used $\{\theta_t^x\}$ and not $\{\theta_t\}$. Four independent checks, each of which distinguishes the two schedules:

Check	Run A	Run B
Training log, line "Noise schedule [train]" written when training starts	`theta_power=1.00`, $\bar{\alpha}_T = 4.386 \times 10^{-5}$	`theta_power=2.50`, $\bar{\alpha}_T = 1.274 \times 10^{-11}$
`alphas_cumprod` buffer saved inside the final checkpoint, compared in double precision with $\bar{\alpha}_t$ and with $\bar{\alpha}_t^{2.50}$	equals $\bar{\alpha}_t$: yes; equals $\bar{\alpha}_t^{2.50}$: no	equals $\bar{\alpha}_t$: no; equals $\bar{\alpha}_t^{2.50}$: yes
Schedule recorded by the test evaluation (`eval_config.json`)	$\bar{\alpha}_1 = 0.9999990$, $\bar{\alpha}_T = 4.386 \times 10^{-5}$	$\bar{\alpha}_1 = 0.9999975$, $\bar{\alpha}_T = 1.274 \times 10^{-11}$
Reverse-process snapshots at t = 1,000 (Figure 6)	image structure already visible ($\bar{\alpha}_t = 0.082$)	still indistinguishable from noise ($\bar{\alpha}_t^x = 0.0019$)

Table 2. Evidence that the two fine-tuning runs used different schedules. The expected values are $\bar{\alpha}_T = 4.386 \times 10^{-5}$ for the native schedule and 1.274×10^{-11} for $x = 2.50$.

In addition, the schedule produced by the model code is compared against an independent double-precision computation in `assignment/schedule_analysis.py`, and the two agree for both exponents.

6. Results

Run 0 outputs. Figure 2 shows the pretrained model applied directly to eight test images.

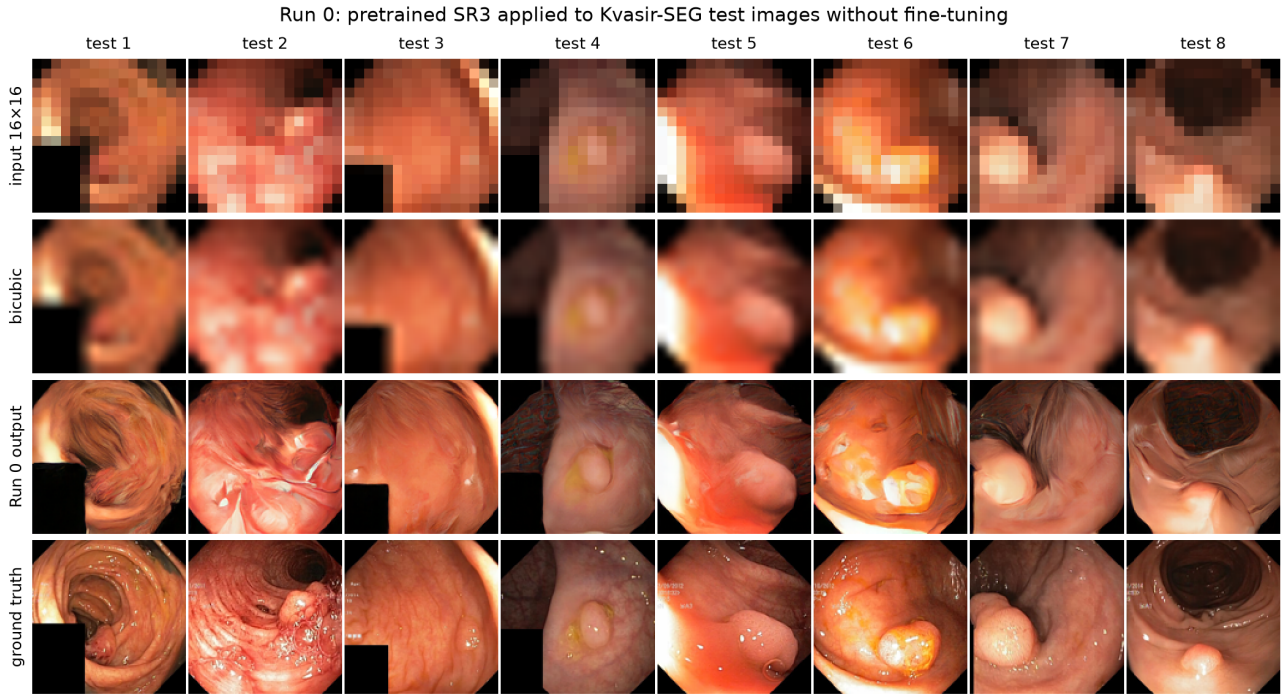


Figure 2. Run 0. Rows: 16×16 input (enlarged), bicubic upsampling, Run 0 output, ground truth. All 100 outputs are in assignment_results/run_0_baseline/test_outputs/sr/.

Fine-tuning loss. The script logged the loss of one batch every 100 iterations, that is two batches per epoch. The value in the submitted CSV files (`epoch, train_loss`) is the mean of those two batches, not the mean over all 200 batches of the epoch, which was not recorded. Because the loss of a batch depends strongly on the timestep it happened to draw, the per-epoch values are noisy, and Figure 3 adds a 10-epoch moving average.

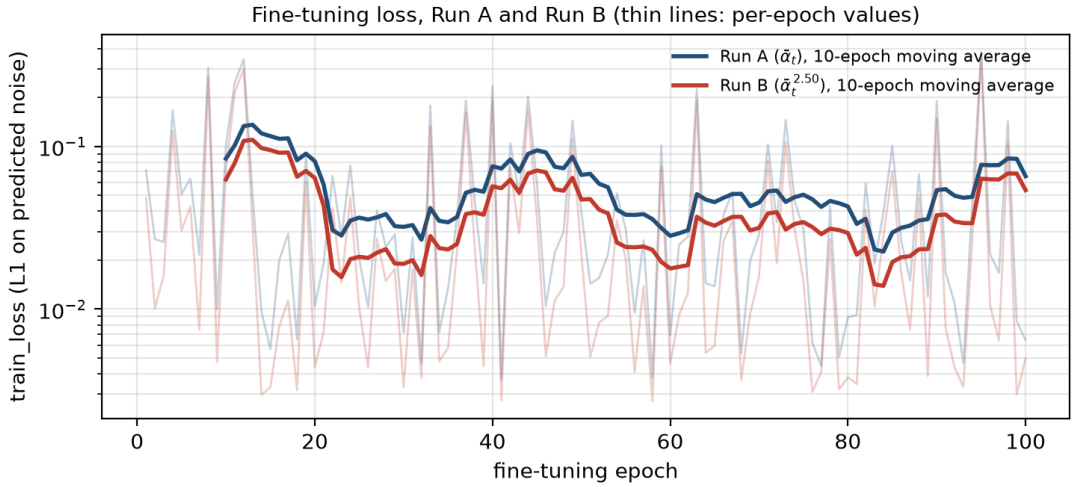


Figure 3. Fine-tuning loss of Run A and Run B on one plot (log scale). Thin lines: per-epoch values from the CSV files. Thick lines: 10-epoch moving average.

The two curves have the same shape because both runs share a seed and therefore draw the same timestep index in every iteration. Run B's averaged curve lies below Run A's throughout (mean over all epochs 0.042 against 0.058). This does not mean Run B learned better: at the same timestep index its input is noisier, and predicting the noise in a noisier image is an easier problem. Loss values are comparable within a run, not across the two schedules.

Samples at five fine-tuning checkpoints. Epochs 20, 40, 60, 80 and 100. The samples are the ones generated by the model at that epoch during validation, for the same three validation images in both runs.

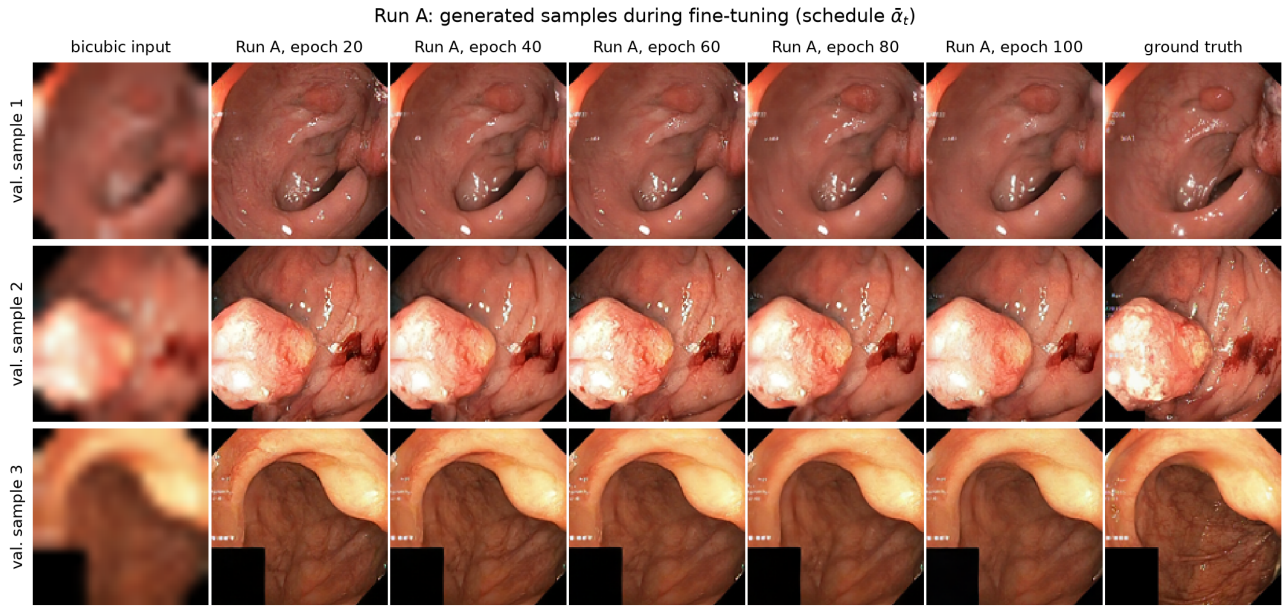


Figure 4. Run A (native schedule), samples at epochs 20 to 100.

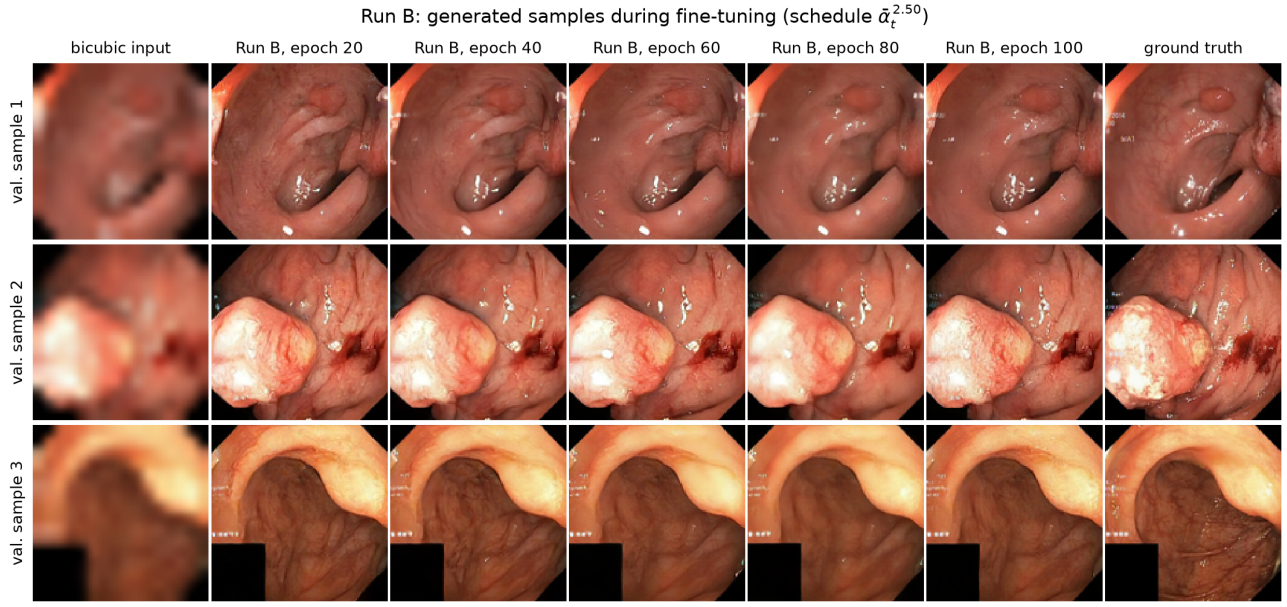


Figure 5. Run B (schedule raised to x = 2.50), samples at epochs 20 to 100.

Epoch	PSNR Run A	PSNR Run B	B − A (± s.e.)	LPIPS Run A	LPIPS Run B	B − A (± s.e.)
20	23.04	22.73	-0.31 ± 0.31	0.183	0.193	+0.011 ± 0.004
40	23.75	23.83	+0.08 ± 0.09	0.181	0.189	+0.008 ± 0.004
60	23.27	23.42	+0.15 ± 0.33	0.188	0.192	+0.004 ± 0.004
80	23.80	23.58	-0.22 ± 0.25	0.182	0.197	+0.016 ± 0.004
100	23.82	22.79	-1.03 ± 0.38	0.186	0.184	-0.003 ± 0.006

Table 3. The two runs at the five checkpoints, on the 20 validation images sampled during fine-tuning. Differences are paired per image; s.e. is the standard error of the mean difference.

Inference trajectories. Noisy sample x_t at t = 1800, 1400, 1000, 600 and 200 during the reverse process of the final models, for three test images. The snapshots come from the same sampling runs that produced the test scores.

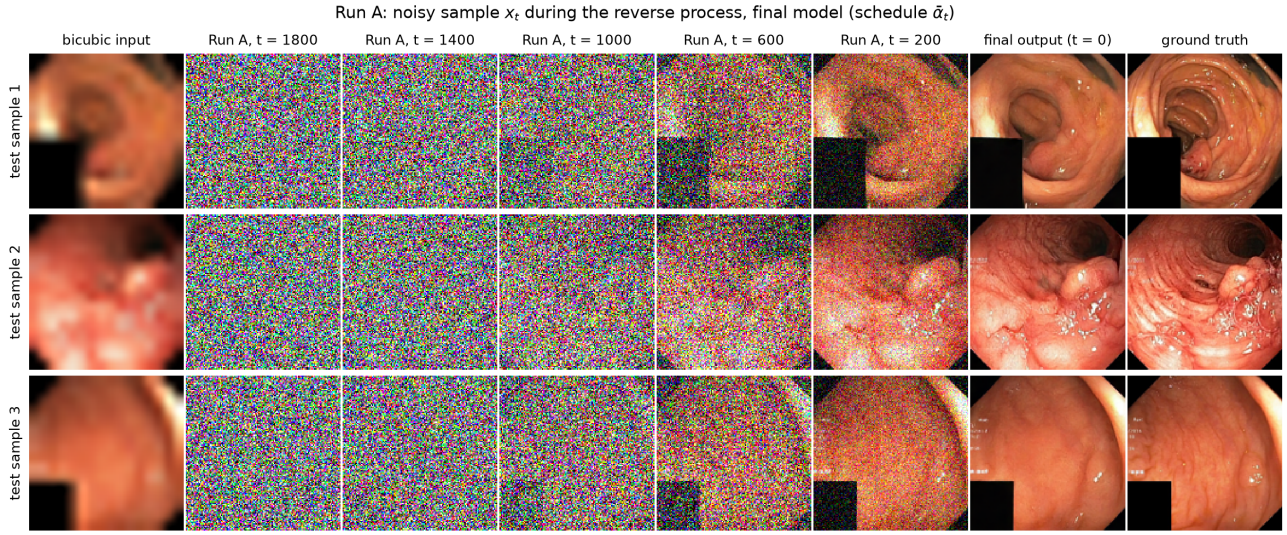


Figure 6a. Run A, final model: x_t at five equally spaced timesteps.

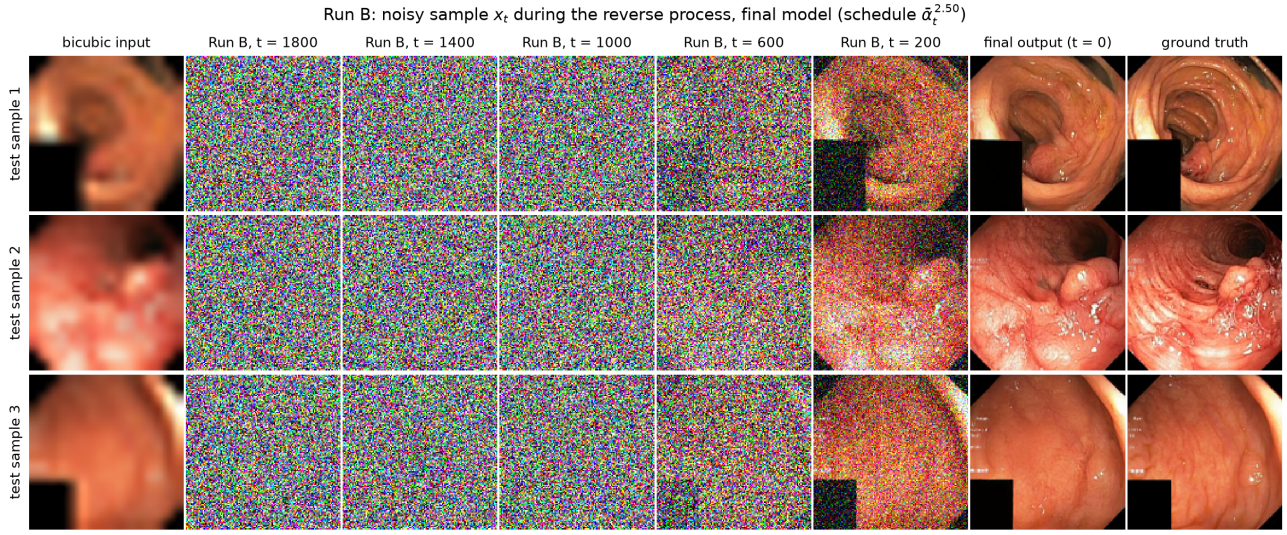


Figure 6b. Run B, final model: x_t at the same timesteps and from the same starting noise.

Test-set scores. Our chosen metric is **PSNR**: in a medical image, pixel-level fidelity to the true frame matters more than a plausible-looking texture, and PSNR measures exactly that. SSIM and LPIPS are reported alongside it.

Method	PSNR (dB) ↑	SSIM ↑	LPIPS ↓
Bicubic upsampling (reference)	23.21 ± 1.50	0.636 ± 0.086	0.560 ± 0.065
Run 0: pretrained, no fine-tuning	**22.49 ± 1.89**	0.572 ± 0.092	0.273 ± 0.061
Run A: fine-tuned, $\bar{\alpha}_t$	**24.15 ± 2.33**	0.614 ± 0.098	0.170 ± 0.053
Run B: fine-tuned, $\bar{\alpha}_t^{2.50}$	**23.43 ± 2.17**	0.598 ± 0.096	0.171 ± 0.046

Table 4. Test-set scores, mean ± standard deviation over the 100 test images. One sample per image, 2,000 sampling steps, same starting noise in all runs.

Comparison	PSNR difference (dB)	Images improved	SSIM difference	LPIPS difference
Run A − Run 0	+1.66 ± 0.18	83 / 100	+0.041 ± 0.004	-0.102 ± 0.004
Run B − Run 0	+0.94 ± 0.20	70 / 100	+0.026 ± 0.005	-0.102 ± 0.005
Run B − Run A	-0.72 ± 0.15	15 / 100	-0.016 ± 0.004	+0.001 ± 0.003

Table 5. Paired per-image differences on the test set, mean ± standard error. "Images improved" counts the images on which the first-named run has the higher PSNR.

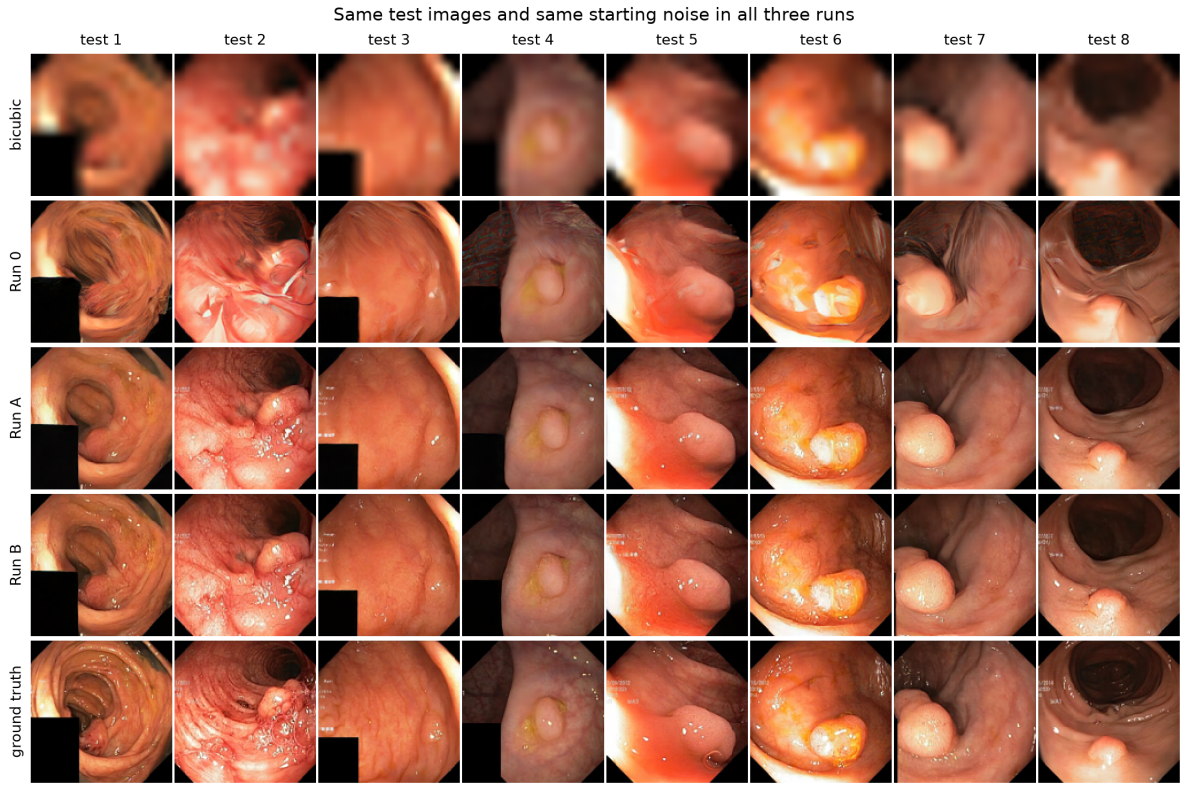


Figure 7. The three runs on the same eight test images.

7. Analysis

Which run is better. On the chosen metric the native schedule wins. Run A reaches 24.15 dB and Run B 23.43 dB, a paired difference of -0.72 ± 0.15 dB, and Run B has the higher PSNR on only 15 of the 100 test images. SSIM agrees (-0.016 ± 0.004). LPIPS does not separate the runs at all (+0.001 ± 0.003, each run better on 50 images). So Run B's outputs look as realistic as Run A's but are less faithful to the true image.

When the difference became visible. By eye, the samples of the two runs in Figures 4 and 5 are hard to tell apart at any of the five checkpoints. The numbers in Table 3 say the same about fidelity: at epochs 20, 40, 60 and 80 the PSNR difference is within about one standard error of zero and changes sign, and only at **epoch 100** does it exceed the noise (-1.03 ± 0.38 dB on validation, matching the test result). Perceptual quality behaves differently. LPIPS already favours Run A at the first sampled checkpoint, epoch 20 (+0.011 ± 0.004), and the gap was largest at the very first validation pass at epoch 10 (LPIPS 0.264 for Run B against 0.201 for Run A). It then shrinks, and at epoch 100 the two runs are level. Our reading is that Run B needed longer to adapt, and that the PSNR gap is something we can only establish at the final checkpoint.

Mechanistic explanation. SR3 conditions on the noise level, not on the timestep index. Changing the schedule therefore does not give the network a different task; it changes how training and sampling effort is distributed over noise levels. Three consequences follow from Section 3.

- **Wasted training steps.** Training draws t uniformly. Under $\bar{\alpha}_t^{2.50}$, 36.8% of the draws land at t ≥ 1266, where the SNR is below 4.4×10^{-5} and x_t is, for practical purposes, pure noise. The best possible prediction there is the input itself, so these updates teach the network nothing about endoscopy images. Run A spends all of its 20,000 updates inside the range where the image still matters; Run B spends about 12,650 of them there.
- **Less practice on fine detail.** Fine texture is decided at low noise. Timesteps with more signal than noise make up 26.3% of the native schedule (526 of 2,000) but only 16.6% of the modified one (333). Run B therefore gets roughly 63% as many updates on the noise levels that fix the final pixel values.
- **Coarser sampling.** The reverse process of Run B spends its first 735 steps where nothing can be decided and then has to cross the whole informative range in the remaining 1,265 steps, with

per-step variance about 2.50 times larger. Figure 6 shows this directly: at t = 1,000 Run A already shows the layout of the image ($\bar{\alpha}_t$ = 0.082) while Run B is still noise (0.0019); at t = 600 Run A is well formed (0.41) and Run B is only starting (0.11). The image is assembled in fewer, larger steps.

The error pattern is consistent with this account. We split each test output's error into a smooth part and a detail part (Gaussian low-pass, σ = 3 pixels, and the remainder). Run B has more error than Run A in both: 60 against 47 in the smooth part and 232 against 206 in the detail part (mean squared error on the 0-255 scale). At the same time its outputs contain more fine detail, 78% of the ground truth's high-frequency energy against 70% for Run A. Run B produces at least as much texture, but less of it is in the right place, which is what equal LPIPS with lower PSNR looks like. A shift in overall colour is not the cause: it accounts for about 1% of the error in every run.

How firm this is. Each run was trained once. Validation PSNR moved by up to a decibel between neighbouring checkpoints in both runs (Run A fell to 22.81 dB at epoch 50; Run B fell from 24.19 dB at epoch 90 to 22.79 dB at epoch 100). We evaluated the epoch-100 checkpoint of both runs because that was fixed in advance, but part of the 0.72 dB test gap may be this checkpoint-to-checkpoint variation rather than the schedule. The direction of the effect agrees with the mechanism above; its size would need repeated seeds to pin down.

Run 0 compared with the fine-tuned runs. Without fine-tuning, the face-trained model scores 22.49 dB, below plain bicubic upsampling (23.21 dB), yet its LPIPS (0.273) is far better than bicubic (0.560). It has learned how to sharpen: its outputs carry 75% of the ground truth's high-frequency energy, where bicubic carries 7%. But the detail it adds comes from faces. In Figure 2 the mucosa is rendered with smooth, skin-like surfaces and invented creases, and fine vessel patterns are missing, so the added detail raises the pixel error instead of lowering it. Fine-tuning corrects exactly this. Run A improves on Run 0 by +1.66 ± 0.18 dB with 83 of 100 images better, and Run B by +0.94 ± 0.20 dB; LPIPS falls from 0.273 to 0.170 in both, with nearly every image improved.

What this says about fine-tuning on an out-of-distribution dataset: the general skill of the pretrained model, turning a blurry input into a sharp image that agrees with it, carries over to a domain it has never seen. What does not carry over is its idea of what the detail should look like, and 800 images and 100 epochs are enough to replace that. The gain from fine-tuning (1.66 dB for Run A) is more than twice the difference between the two schedules (0.72 dB), so here adapting to the domain mattered more than the choice of schedule. Even after fine-tuning, SSIM stays below bicubic (0.614 against 0.636). At 8× from a 16×16 input the problem is heavily under-determined, and a single sharp sample is penalised by pixel-wise metrics relative to a blurry average.

8. Takeaway

- The power transformation is not a small perturbation. For this schedule, raising $\bar{\alpha}_t$ to x = 2.50 is the same as running the native process 1.58 times faster: about 37% of the timesteps end up at noise levels the pretrained model never saw and that carry no information about the image.
- The model coped with this better than we expected. Because SR3 is conditioned on the noise level, the modified schedule reuses what was learned in pretraining instead of invalidating it, and Run B still improved clearly on Run 0. The cost showed up in fidelity (0.72 dB) and not in perceptual quality.
- A lower training loss did not mean a better model. Run B's loss was lower throughout only because its denoising problems were easier on average. Loss curves under different schedules cannot be compared directly.
- Fine-tuning mattered more than the schedule. A model trained only on faces transferred its sharpening ability to endoscopy at once, but needed fine-tuning to stop producing face-like texture.
- Checking the mechanics was worth the effort. The checkpoint silently carries its own schedule, and without the fix in `model.py` and the checks in Table 2 we could not have been sure which schedule Run B really used.

- One seed per run and one sample per image leave real uncertainty. Validation scores swung by a decibel between checkpoints, so with more time we would repeat both runs with several seeds and average over several samples before making a stronger claim about the size of the gap.

9. Contribution Statement

Member	Roll number	Responsible for	Signature
Rikin Patel	M26AI1096	[COMPONENT(S) - to be filled in]	
Anshul	M26AI1020	[COMPONENT(S) - to be filled in]	
Sonalika Sharma	M26AI1120	[COMPONENT(S) - to be filled in]	
Balasubramanium M	M26AI1030	[COMPONENT(S) - to be filled in]	

We confirm that this report describes our own work and that the contributions listed above are accurate.

Appendix: submission contents

Component	Location in the repository
Modified code and README	repository root, branch `feature/phase2-report`; exponent constant in `sr.py`, line 3; reproduction commands, seed and library versions in `README.md`
Fine-tuning logs	`assignment_results/phase2/finetuning_logs/runA_train_loss.csv` and `runB_train_loss.csv`; full logs in `assignment_results/run_*/logs/`
Checkpoint samples	`assignment_results/phase2/checkpoint_samples/`, files named `runA_epoch020_sample1.png` etc., each with a caption bar
Inference trajectories	`assignment_results/phase2/inference_trajectories/`, files named `runB_t1400_sample2.png` etc.; all eleven snapshots per image in `assignment_results/run_*/trajectories/`
Run 0 outputs	`assignment_results/run_0_baseline/test_outputs/sr/` (100 images)
Scores	`metrics_per_image.csv` and `metrics_summary.json` in each run folder; `assignment_results/comparison/`